Contents

Module 5 — Introduction to VBA and Linear Macros

Learning outcomes

Students will be able to:

- identify Excel components addressed by VBA;
- open the Visual Basic Editor and interpret a `Sub` structure;
- explain compile time, runtime, statements, and errors at an introductory level;
- record spreadsheet operations as a macro;
- read and simplify recorded code; and
- create a new linear program without the recorder.

100-minute sequence

Minutes	Activity
0–10	Trace Workbook–Worksheet–Range objects
10–25	Visual Basic Editor, modules, `Sub`, and statements
25–42	Record an Excel operation
42–57	Read and simplify the recording
57–75	Build a linear concrete-volume macro
75–84	Compile, run, and classify errors
84–97	Three-question short exercise
97–100	Save `.xlsm` and complete checklist

Minimum product: one simplified recorded macro and one original linear macro that reads input, calculates, and writes output.

1. Excel's object hierarchy

```
Application
└── Workbook
    └── Worksheet
        └── Range/Cells
```

An explicit reference is:

```
ThisWorkbook.Worksheets("Volume").Range("B2").Value
```

- `ThisWorkbook` is the workbook containing the code.
- `Worksheets("Volume")` selects a named sheet.
- `Range("B2")` selects a cell.
- `.Value` reads or changes its value.

Explicit references are more reliable than depending on whichever workbook or sheet happens to be active.

2. Set up VBA

1. Save the workbook as `.xlsm`.
2. Enable the Developer tab.
3. Press `Alt+F11`.
4. Select `Insert` → `Module`.
5. Put `Option Explicit` on the first line.

```
Option Explicit

Sub ProcedureName()
    ' statements execute from top to bottom
End Sub
```

A statement is one instruction. An apostrophe begins a comment.

3. Compile time, runtime, and errors

- **Compile/check:** VBA checks syntax and declarations. Use `Debug` → `Compile VBAProject`.
- **Runtime:** statements execute using actual values and objects.
- **Syntax/compile error:** the code violates a language rule.
- **Runtime error:** execution begins and then fails, for example because a sheet is missing.
- **Logic error:** execution completes but the result is wrong.

At this level, the important distinction is when an error becomes visible: while checking code, while running it, or only after validating the result.

4. Macro Recorder

Record this workflow:

1. start `Record Macro` and name it `FormatHeader`;
2. enter `Length (m)`, `Width (m)`, and `Volume (m³)` in A1:C1;
3. make the text bold with a grey background;
4. stop recording; and
5. inspect the generated code.

The recorder may use `Select` and `Selection`:

```
Range("A1:C1").Select
Selection.Font.Bold = True
Selection.Interior.Color = RGB(217, 217, 217)
```

A more direct form is:

```
With ThisWorkbook.Worksheets("Volume").Range("A1:C1")
    .Font.Bold = True
```

```
    .Interior.Color = RGB(217, 217, 217)
End With
```

The recorder helps discover object, property, and method names. Its output is a starting point that should be understood and cleaned up.

5. A linear program

Prepare a sheet named `Volume`:

Cell	Content
A2/B2	Length (m) / 10
A3/B3	Width (m) / 2
A4/B4	Height (m) / 0.3
A6	Volume (m³)

```
Option Explicit

Sub CalculateLinearVolume()
    Dim length_m As Double
    Dim width_m As Double
    Dim height_m As Double
    Dim volume_m3 As Double
    Dim ws As Worksheet

    Set ws = ThisWorkbook.Worksheets("Volume")

    length_m = ws.Range("B2").Value
    width_m = ws.Range("B3").Value
    height_m = ws.Range("B4").Value

    volume_m3 = length_m * width_m * height_m

    ws.Range("B6").Value = volume_m3
    ws.Range("B6").NumberFormat = "0.000"
End Sub
```

The reference result is `6.000 m³`.

6. Modify the program

Add a 5% waste factor:

```
Const WASTE_FACTOR As Double = 1.05
Dim orderVolume_m3 As Double

orderVolume_m3 = volume_m3 * WASTE_FACTOR
ws.Range("B7").Value = orderVolume_m3
```

Test after every small change rather than allowing many untested changes to accumulate.

Short exercise — 3 questions

1. Predict

The recorder produces `Range("B2").Select`, followed by `Selection.Value = 10`. What is the risk if another worksheet is active? Write a version that does not depend on selection.

2. Fix

The volume macro returns `60 m³` for 10 m × 2 m × 0.3 m. Use a trace table to identify one plausible logic/data error and repair it.

3. Explain

Classify these events: `End Sub` is missing; worksheet `Volume` does not exist; addition is used instead of multiplication. Which is a syntax/compile error, runtime error, and logic error?

1. B2 on the active sheet may be changed. Use `ThisWorkbook.Worksheets("Volume").Range("B2").V = 10`.
2. Examples include reading height as `3` instead of `0.3`, or failing to convert centimetres. The trace table must reveal the actual input values.
3. Missing `End Sub`: syntax/compile; missing worksheet: runtime; addition instead of multiplication: logic.

Checklist

- ☐ The workbook is saved as `.xlsm`.
- ☐ I can trace Workbook–Worksheet–Range.
- ☐ I understand every important line retained from the recorder.
- ☐ The linear macro agrees with a manual calculation.

Further reading

1. Michael Alexander & Dick Kusleika, *Excel 2019 Power Programming with VBA*, Wiley, 2019.
2. Bill Jelen & Tracy Syrstad, *Microsoft Excel VBA and Macros (Office 2021 and Microsoft 365)*, Microsoft Press, 2022.
3. Steven Roman, *Writing Excel Macros with VBA*, 2nd ed., O'Reilly Media, 2002.
4. Bernard Liengme & Keith Hekman, *Liengme's Guide to Excel 2016 for Scientists and Engineers*, Academic Press, 2019.

← Module 4 · Module list · Module 6 →